Python Dictionary

A dictionary in Python is a collection of key-value pairs. It is unordered, mutable, and indexed by keys. Dictionaries are written using curly braces {} and store data in key-value pairs.

1. Creating a Dictionary

dictionary_name = {key1: value1, key2: value2, ...}

Example:

```
student = {
    "name": "Alice",
    "age": 25,
    "course": "Data Science"
}
print(student)

# Output: {'name': 'Alice', 'age': 25, 'course': 'Data Science'}
```

2. Accessing Dictionary Elements

You can access values using keys.

Example:

```
print(student["name"])  # Output: Alice
print(student.get("age"))  # Output: 25
```

.get() method is safer because it returns None if the key doesn't exist, whereas using [] will cause an error.

3. Adding and Updating Values

Adding a new key-value pair:

```
student["grade"] = "A"
print(student)
# Output: {'name': 'Alice', 'age': 25, 'course': 'Data Science', 'grade': 'A'}
```

Updating an existing value:

```
student["age"] = 26
print(student)
# Output: {'name': 'Alice', 'age': 26, 'course': 'Data Science', 'grade': 'A'}
```

4. Removing Elements from a Dictionary

Using .pop(key) – Removes a key and returns its value

```
removed_value = student.pop("grade")
print(removed_value)  # Output: A
print(student)
# Output: {'name': 'Alice', 'age': 26, 'course': 'Data Science'}
```

Using del – Deletes a key-value pair

```
del student["course"]
print(student)
# Output: {'name': 'Alice', 'age': 26}
```

Using .popitem() – Removes and returns the last inserted item

```
student.popitem()
print(student)
# Output: {'name': 'Alice'}
```

Using .clear() – Removes all elements

```
student.clear()
print(student)  # Output: {}
```

5. Looping Through a Dictionary

Iterating through keys:

```
for key in student:
    print(key)
# Output:
# name
# age
```

Iterating through values:

```
for value in student.values():
    print(value)
# Output:
# Alice
# 26
```

Iterating through key-value pairs:

```
for key, value in student.items():
    print(f"{key}: {value}")
```

```
# Output:
# name: Alice
# age: 26
```

6. Dictionary Methods

Method	Description
dict.get(key, default)	Returns value of key, or default if key doesn't exist
dict.keys()	Returns a list of dictionary keys
dict.values()	Returns a list of dictionary values
dict.items()	Returns a list of key-value pairs (tuples)
dict.pop(key)	Removes a key and returns its value
dict.popitem()	Removes and returns the last inserted key-value pair
dict.update(other_dict)	Merges another dictionary into the current one
dict.clear()	Removes all key-value pairs
dict.copy()	Creates a shallow copy of the dictionary

7. Dictionary Comprehension

Example: Creating a dictionary with squares of numbers

```
squares = {x: x**2 for x in range(1, 6)}
print(squares)
# Output: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

8. Nested Dictionary

A dictionary inside another dictionary.

Example:

```
students = {
    "Alice": {"age": 25, "course": "Data Science"},
    "Bob": {"age": 23, "course": "Python"}
```

```
}
print(students["Alice"]["course"])  # Output: Data Science
```

9. Checking if a Key Exists

```
if "name" in student:
    print("Key exists!")
```

10. Merging Two Dictionaries

```
dict1 = {"a": 1, "b": 2}
dict2 = {"c": 3, "d": 4}
merged_dict = {**dict1, **dict2}
print(merged_dict)
# Output: {'a': 1, 'b': 2, 'c': 3, 'd': 4}
```

notes:

Dictionaries store key-value pairs.

Keys must be unique & immutable (strings, numbers, tuples).

Values can be any data type.

Dictionaries are unordered (Python 3.6+ maintains insertion order).

Use .get() to avoid key errors.

Use dictionary methods for efficient operations.

Python Tuples

A tuple in Python is an ordered, immutable collection of items. Unlike lists, tuples cannot be modified (no adding, removing, or updating elements). Tuples are defined using parentheses ().

1. Creating a Tuple

Syntax:

tuple_name = (value1, value2, value3, ...)

Examples:

```
# Tuple with multiple values
fruits = ("apple", "banana", "cherry")
print(fruits)  # Output: ('apple', 'banana', 'cherry')

# Tuple with different data types
person = ("Alice", 25, True)
print(person)  # Output: ('Alice', 25, True)

# Single-element tuple (must include a comma!)
single_tuple = ("hello",)
print(single_tuple)  # Output: ('hello',)

# Without a comma, Python treats it as a string, not a tuple
not_a_tuple = ("hello")
print(type(not_a_tuple))  # Output: <class 'str'>

# Empty tuple
empty_tuple = ()
print(empty_tuple)  # Output: ()
```

2. Accessing Tuple Elements

Elements in a tuple are accessed using indexing, starting from 0.

```
fruits = ("apple", "banana", "cherry")

print(fruits[0])  # Output: apple
print(fruits[1])  # Output: banana
print(fruits[-1])  # Output: cherry (negative index for last element)
```

3. Slicing a Tuple

```
numbers = (10, 20, 30, 40, 50)

print(numbers[1:4])  # Output: (20, 30, 40)  [Index 1 to 3]
print(numbers[:3])  # Output: (10, 20, 30)  [Start to index 2]
print(numbers[2:])  # Output: (30, 40, 50)  [Index 2 to end]
print(numbers[::-1])  # Output: (50, 40, 30, 20, 10) [Reverse tuple]
```

4. Tuple Operations

Concatenation

```
tuple1 = (1, 2, 3)
tuple2 = (4, 5, 6)

new_tuple = tuple1 + tuple2
print(new_tuple)  # Output: (1, 2, 3, 4, 5, 6)
```

Repetition

```
repeat_tuple = ("Hello",) * 3
print(repeat_tuple)  # Output: ('Hello', 'Hello', 'Hello')
```

Membership Test

```
fruits = ("apple", "banana", "cherry")
print("apple" in fruits)  # Output: True
print("grape" in fruits)  # Output: False
```

5. Tuple Methods

Tuples have only two built-in methods:

Method	Description
tuple.count(value)	Returns the number of times value appears in the tuple
tuple.index(value)	Returns the index of the first occurrence of value

Examples:

```
numbers = (1, 2, 3, 4, 2, 5, 2)

print(numbers.count(2))  # Output: 3 (2 appears three times)
print(numbers.index(4))  # Output: 3 (4 is at index 3)
```

6. Tuple Immutability

Tuples cannot be modified after creation.

```
fruits = ("apple", "banana", "cherry")
fruits[0] = "grape"  # Error: Tuples are immutable!
```

Workaround: Convert Tuple to List

If modification is necessary, convert the tuple to a list first.

```
fruits = ("apple", "banana", "cherry")
```

```
fruits_list = list(fruits)  # Convert to list
fruits_list[0] = "grape"  # Modify the list
fruits = tuple(fruits_list)  # Convert back to tuple
print(fruits)  # Output: ('grape', 'banana', 'cherry')
```

7. Packing and Unpacking Tuples

Packing (Storing multiple values in a tuple)

```
person = ("Alice", 25, "Engineer")
```

Unpacking (Extracting values into variables)

```
name, age, profession = person
print(name)  # Output: Alice
print(age)  # Output: 25
print(profession)  # Output: Engineer
```

Using * for Variable-Length Unpacking

```
numbers = (1, 2, 3, 4, 5)

a, b, *rest = numbers
print(a)  # Output: 1
print(b)  # Output: 2
print(rest)  # Output: [3, 4, 5] (remaining values as a list)
```

8. Looping Through a Tuple

```
fruits = ("apple", "banana", "cherry")

for fruit in fruits:
```

```
    print(fruit)
# Output:
# apple
# banana
# cherry
```

9. Nested Tuples

A tuple can contain other tuples (nested tuples).

```
nested_tuple = ((1, 2, 3), ("a", "b", "c"))
print(nested_tuple[1][2])  # Output: c
```

Tuple vs List (Key Differences)

Feature	Tuple	List
Mutability	Immutable (cannot change)	Mutable (can change)
Syntax	`()`	`[]`
Methods	Limited (only `.count()` and `.index()`)	Many methods available
Performance	Faster	Slower
Memory Usage	Uses less memory	Uses more memory
Use Case	Fixed data (e.g., coordinates, database records)	Dynamic data (e.g., user input, to-do lists)

11. Real-World Example of Tuples

Storing Latitude & Longitude (Immutable Data)

```
location = (40.7128, -74.0060)  # New York City (Latitude, Longitude)
print(f"Location: {location[0]}, {location[1]}")
# Output: Location: 40.7128, -74.0060
```

Returning Multiple Values from a Function

```
def get_employee_details():
    return ("Alice", "Software Engineer", 80000)

name, job, salary = get_employee_details()
print(name, job, salary)
# Output: Alice Software Engineer 80000
```

Note:

Tuples are immutable (cannot be modified).

Defined using () and store ordered collections of elements.

Access elements using indexing & slicing.

Use .count() and .index() methods.

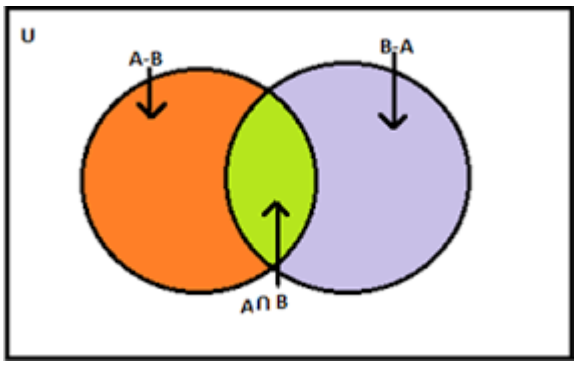
Support unpacking & nesting.

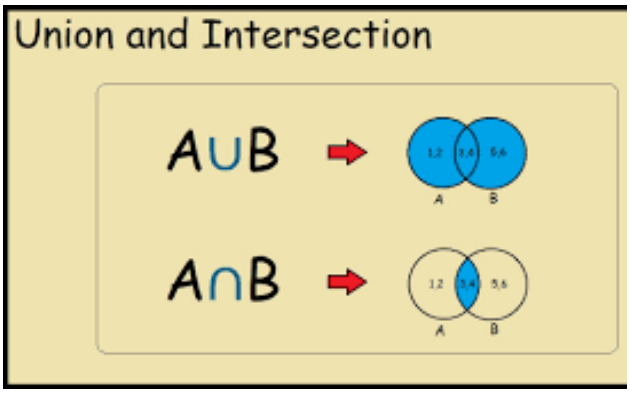
Faster and more memory-efficient than lists.

Best for fixed data like coordinates, database records, etc.

Python Set

In Python, a set is an unordered collection of unique elements. It is defined using curly braces {} or the set() function.





Creating a Set

```
# Using curly braces
my_set = {1, 2, 3, 4, 5}
print(my_set)  # Output: {1, 2, 3, 4, 5}

# Using the set() function
another_set = set([3, 4, 5, 6, 7])
print(another_set)  # Output: {3, 4, 5, 6, 7}

# Creating an empty set
empty_set = set()  # {} creates an empty dictionary, not a set
print(type(empty_set))  # Output: <class 'set'>
```

Set Operations

Python sets support various operations like union, intersection, and difference.

```
A = {1, 2, 3, 4}
```

```
B = {3, 4, 5, 6}

# Union (A ∪ B)
print(A | B)  # Output: {1, 2, 3, 4, 5, 6}
print(A.union(B))

# Intersection (A ∩ B)
print(A & B)  # Output: {3, 4}
print(A.intersection(B))

# Difference (A - B)
print(A - B)  # Output: {1, 2}
print(A.difference(B))

# Symmetric Difference (A Δ B)
print(A ^ B)  # Output: {1, 2, 5, 6}
print(A.symmetric_difference(B))
```

Modifying a Set

```
# Adding elements
A.add(5)
print(A)  # Output: {1, 2, 3, 4, 5}

# Removing elements
A.remove(3)  # Raises an error if 3 is not present
A.discard(10)  # Does not raise an error if 10 is not present

# Popping a random element
A.pop()
```

Clearing all elements

A.clear()

Set Comprehension

```
squared_set = {x**2 for x in range(6)}

print(squared_set)  # Output: {0, 1, 4, 9, 16, 25}
```

How to Access Elements in a Python set

Since sets are unordered and unindexed, you cannot access elements using an index like lists. Instead, you can access set elements using loops, membership tests, or by converting to a list.

1. Using a Loop (for loop)

```
my_set = {10, 20, 30, 40, 50}

for item in my_set:
    print(item)  # Prints elements one by one
```

2. Checking if an Element Exists (in keyword)

```
print(20 in my_set)  # Output: True

print(100 in my_set) # Output: False
```

3. Converting a Set to a List (For Indexing)

```
my_list = list(my_set)

print(my_list[0])  # Accessing the first element
```

Note: The order may not be the same every time because sets are unordered.

4. Using pop() to Get and Remove an Element

```
element = my_set.pop()  # Removes a random element
print(element)
print(my_set)
```

Looping Through a Set in Python

Since sets are unordered, you can iterate over them using a for loop.

Example: Loop Through a Set

```
my_set = {1, 2, 3, 4, 5}

for item in my_set:
    print(item)
```

Output (Order may vary):

```
1
2
3
4
5
```

Using a while Loop

```
my_set = {10, 20, 30}
while my_set:
    print(my_set.pop())  # Removes and prints elements one by one
```

Joining Sets in Python

You can join two or more sets using union (|) or update (update()).

1. Using union() (|) - Returns a New Set

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}

new_set = set1.union(set2)  # OR set1 | set2
print(new_set)  # Output: {1, 2, 3, 4, 5}
```

2. Using update() - Modifies Original Set

```
set1.update(set2)
print(set1)  # Output: {1, 2, 3, 4, 5}
```

3. Using |= Operator (Shortcut for update())

```
set1 |= set2
print(set1)  # Output: {1, 2, 3, 4, 5}
```